

In Next.js middleware, you can retrieve the country code using the built-in `geo` object or through platform-specific headers if you are using a provider like Vercel or Cloudflare. [🔗](#)

1. Using the Built-in `geo` Object

Next.js provides a `geo` property on the `NextRequest` object that contains geographical information. [🔗 GitHub +1](#)

typescript

```
import { NextRequest, NextResponse } from 'next/server';

export function middleware(req: NextRequest) {
  // Get the country code (e.g., "US", "GB")
  const country = req.geo?.country || 'US';

  // Example: Rewrite to a country-specific route
  const { nextUrl: url } = req;
  url.pathname = `/${country}${url.pathname}`;

  return NextResponse.rewrite(url);
}
```

2. Using Provider-Specific Headers

If the `geo` object is empty (common in local development), you can fall back to headers provided by your deployment platform. [🔗 JavaScript in Plain English +2](#)

- **Vercel:** Use the `x-vercel-ip-country` header.
- **Cloudflare:** Use the `cf-ipcountry` header. [🔗 Vercel Community +2](#)

typescript

```
export function middleware(req: NextRequest) {
  const country = req.headers.get('x-vercel-ip-country') || 'US';
  return NextResponse.next();
}
```

Important Considerations

- **Local Development:** The `req.geo` object and platform headers are usually **not available** when running locally. You should provide a default fallback value for local testing.

- **Middleware Performance:** Keep logic lightweight, as middleware runs on every request that matches your matcher configuration.
- **Next.js 16 Update:** Note that in [Next.js 16](#), Middleware has been renamed to **Proxy**, though functionality for geolocation remains similar.